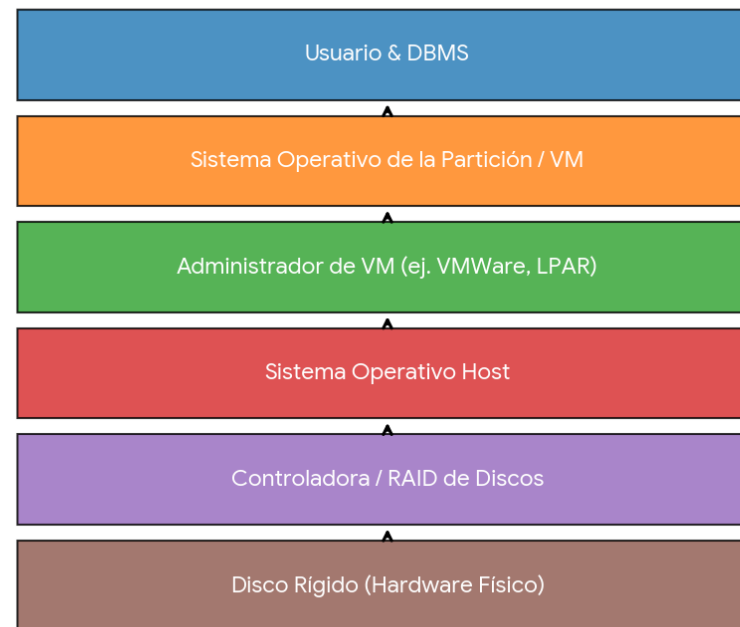


El motor de base de datos (DBMS) está completamente aislado del hardware físico.

Cuando el DBMS necesita guardar un dato, esa petición atraviesa una enorme pirámide de abstracción

- Sale del usuario hacia el DBMS.
- Baja al Sistema Operativo de la partición o máquina virtual.
- Atraviesa el hipervisor o administrador de máquinas virtuales (como VMware o LPARs).
- Llega al Sistema Operativo Host.
- Pasa a la controladora de arreglos (como un RAID de discos).
- Y recién ahí llega a la superficie del disco físico.

Capas de Abstracción de Almacenamiento
El mito del control físico

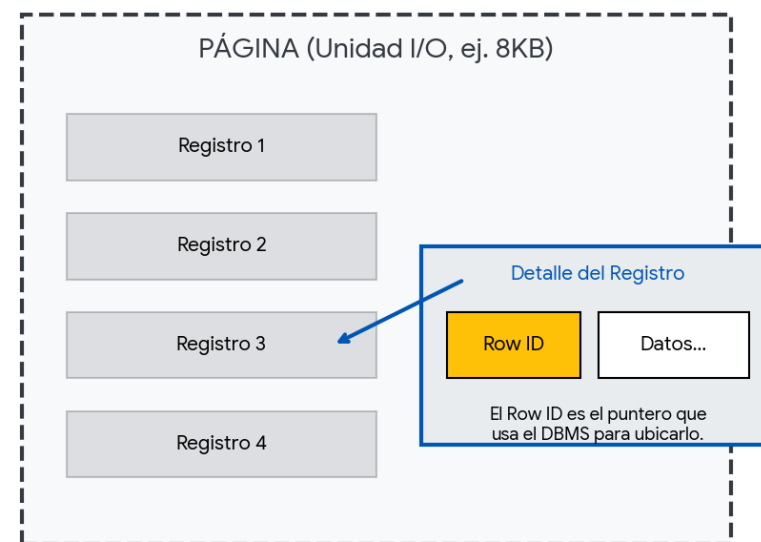


Un DBMS moderno asume un control total, pero en realidad trabaja administrando archivos lógicos (File Systems) que un sistema operativo le cede a ciegas.

En ingeniería de bases de datos no se lee "un registro" del disco duro. La unidad básica de transferencia I/O se llama "página" (o bloque de memoria), y por cuestiones de arquitectura siempre es una potencia de dos, típicamente de 4KB u 8KB.

- Dentro de esa página de 8KB pueden entrar decenas de registros pequeños.
- Para ubicar físicamente el dato, el motor no usa la Clave Primaria (PK) que los desarrolladores definen a nivel lógico. Utiliza un identificador de abstracción interno llamado **Row Id**. Este Row Id actúa como un mapa del tesoro: es el puntero exacto que le dice al motor en qué página de qué archivo está alojado ese registro.

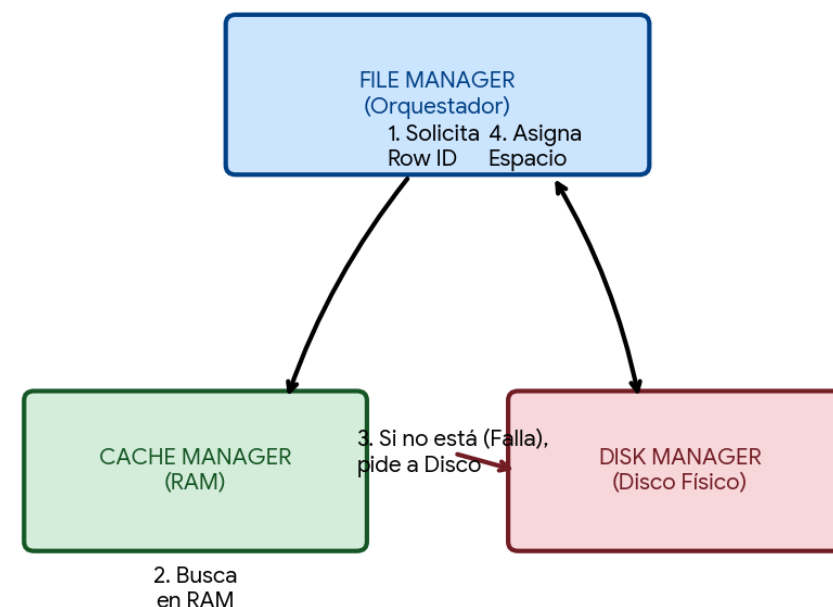
Unidad de Transferencia Física



Para orquestar el viaje de la página desde el plato magnético hasta el procesador, el DBMS implementa una arquitectura interna con tres "Managers":

- **Disk Manager (El guardián del disco):** Administra el espacio a nivel sistema operativo. Sabe exactamente qué bloques de disco están libres u ocupados para poder asignar espacio cuando entran nuevos registros.
- **Cache Manager (El guardián de la RAM):** Las bases de datos viven de evitar el acceso a disco (porque es lento). Este manager reserva espacio en la memoria RAM para almacenar temporalmente las páginas de datos que más se están consultando.
- **File Manager (El orquestador):** Es el nexo de todo. Si una consulta pide un registro, el File Manager obtiene el *Row Id* y se lo pide al Cache Manager. Si el Cache Manager no lo tiene en memoria, recién ahí se realiza el I/O para ir a buscar esa página específica al disco.

Arquitectura de Servicios Internos



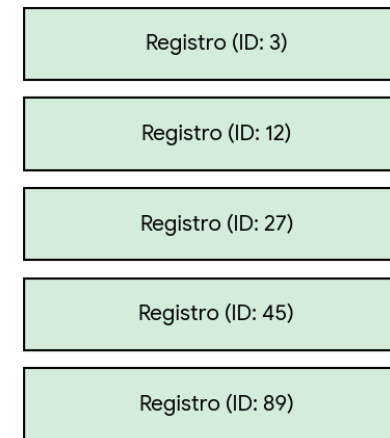
Existen 2 formas para guardar estructuralmente los registros dentro de esas páginas:

- **Archivos Secuenciales (Heap o Montículo):** Los registros "caen" en las páginas en el orden cronológico exacto en que se ejecutan los INSERT. No hay orden lógico.
- **Archivos Agrupados (Clustered):** Los registros se ordenan físicamente en las páginas de disco respetando una regla (por ejemplo, el orden alfabético o numérico de un atributo).

HEAP (Montículo)
Sin ordenamiento físico



CLUSTERED (Agrupado)
Ordenado físicamente por Atributo



Corolario: ¡Solo puede existir 1 índice
Clustered por tabla!

El **Sizing** (o dimensionamiento), es el proceso técnico y analítico mediante el cual se estiman los recursos de infraestructura necesarios para que una base de datos o sistema informático soporte una carga de trabajo proyectada con un rendimiento óptimo.

- **Dimensionamiento de Almacenamiento (Storage):** Calcular cuánto espacio físico en disco ocuparán los datos crudos, los índices, los registros de transacciones (*logs*) y los respaldos (*backups*).
- **Dimensionamiento de Memoria (RAM):** Estimar el tamaño necesario para el *Cache Manager*. El objetivo es asegurar que haya suficiente RAM para mantener los datos de mayor acceso en memoria, evitando lecturas lentas desde el disco físico.
- **Dimensionamiento de Procesamiento (CPU):** Proyectar la potencia de cálculo requerida para manejar la cantidad de usuarios o aplicaciones conectadas simultáneamente y la complejidad de las consultas que van a ejecutar.

COMO CALCULAR EL SIZING

Paso 1: Calcular el tamaño lógico de la fila (Registro)

Sumar el peso en bytes de los tipos de datos definidos en la estructura de la tabla.

- ID_Transaccion (INT) = 4 bytes
- Fecha (DATETIME) = 8 bytes
- ID_Cliente (INT) = 4 bytes
- Monto (DECIMAL/FLOAT) = 8 bytes
- Estado (CHAR(1)) = 1 byte

Total Lógico: 25 bytes

A esto hay que sumarle el **overhead del sistema** (cabeceras de fila, control de nulos, etc.).

Supongamos un overhead estándar de 7 bytes.

Total Físico por Fila: 32 bytes

COMO CALCULAR EL SIZING (cont.)

Paso 2: Calcular la densidad de la página (Rows per Page)

Como vimos en la teoría, los datos se guardan en páginas de 8 KB. Sin embargo, no todo ese espacio es usable (la página en sí tiene una cabecera). Si la página tiene 8192 bytes, el espacio útil suele ser de 8060 bytes.

Para saber cuántos registros entran por página, usamos la siguiente fórmula (redondeando hacia abajo, ya que no podemos guardar fracciones de registros):

$$\text{Filas por Página} = \left\lfloor \frac{\text{Espacio Útil}}{\text{Tamaño Físico de la Fila}} \right\rfloor$$

$$\text{Filas por Página} = \left\lfloor \frac{8060}{32} \right\rfloor = 251 \text{ filas}$$

COMO CALCULAR EL SIZING (cont.)

Paso 3: Calcular el total de páginas necesarias

Ahora cruzamos la densidad de la página con la volumetría proyectada por el negocio (10,000,000 de transacciones).

Acá redondeamos hacia arriba, porque si sobra un registro, necesitamos una página entera nueva.

$$\text{Páginas Totales} = \left\lceil \frac{\text{Volumén de filas}}{\text{Filas por Página}} \right\rceil$$

$$\text{Páginas Totales} = \left\lceil \frac{10,000,000}{251} \right\rceil = 39,841 \text{ Páginas}$$

COMO CALCULAR EL SIZING (cont.)

Paso 4: Traducir páginas a Megabytes (MB)

Finalmente, multiplicamos la cantidad de páginas por su peso real en disco (8 KB) para obtener el tamaño final:

$$\text{Tamaño (KB)} = 39,841 \times 8 = 318,728 \text{ KB}$$

Para llevarlo a Megabytes, lo dividimos por 1024:

$$\text{Tamaño (MB)} = \frac{318,728}{1024} \approx 311.26 \text{ MB}$$